Software Engineering Lab 5: Static Code Analysis

Experiment: Static Code Analysis using Pylint, Bandit and Flake8

Project: Campus Shuttle System

NAME : KARANAM SREEHITHA SECTION : C SRN:PES1UG24AM129

1. Objective

The objective of this lab is to analyse a Python program using static code analysis tools and improve its code quality, security and formatting.

The following tools were used:

- **Pylint:** To identify programming errors, code quality issues and violations of Python coding conventions.
- **Bandit:** To detect potential security-related issues in the Python code.
- **Flake8:** To identify formatting errors and violations of Python style guidelines.

The identified issues were corrected, and the tools were executed again to verify the improvements.

2. Project Description

The Campus Shuttle System is a Python-based program used to maintain basic shuttle route and passenger information.

The program includes functionalities such as:

- Adding passenger trip records.
- Calculating the average passenger load for a route.
- Identifying underused shuttle routes.
- Assigning drivers to routes.
- Saving and retrieving fleet information using JSON files.
- Removing routes and checking route details.

The original program was analysed using all three static code analysis tools before making the required corrections.

3. Tools and Technologies Used

Tool	Purpose
Python 3.11.9	Programming language
Pylint	Code quality analysis
Bandit	Security analysis
Flake8	Code style and formatting analysis
Visual Studio Code	Development environment
Git and GitHub	Version control and submission

4. Initial Analysis

The original Python program was executed and analysed using Pylint, Bandit and Flake8.

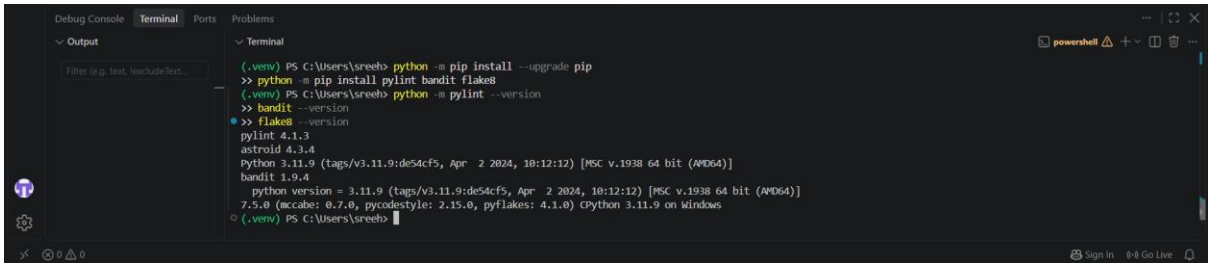
The initial analysis revealed multiple issues, including missing documentation, mutable default arguments, broad exception handling, naming convention violations, potential security concerns and formatting errors.

4.1 Pylint – Initial Analysis

Pylint identified issues such as:

- Missing module and function docstrings.
- Mutable default argument in a function.
- Use of the older string formatting method.
- Bare exception handling.
- Function naming convention violations.
- Unused function arguments.
- Global statement usage.
- Files opened without specifying an encoding.
- Files opened without using a context manager.

Initial Pylint Score: 6.25/10

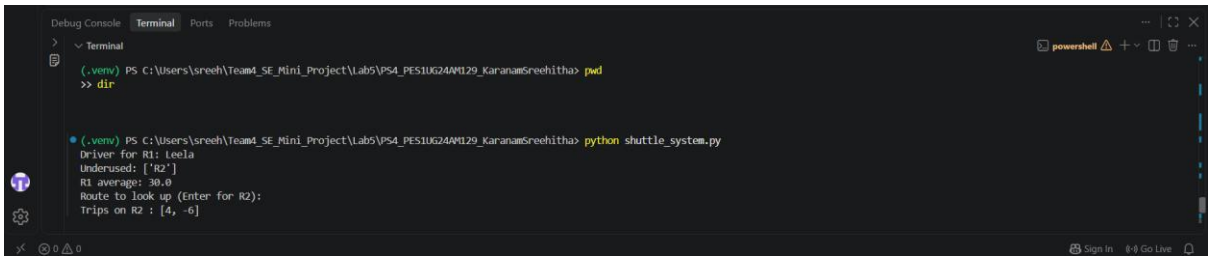


4.2 Bandit – Initial Security Analysis

Bandit identified two low-severity security-related findings:

- **B110 – Try, Except, Pass:** The program contained an exception handler that silently ignored errors.
- **B311 – Standard Pseudo-Random Generators:** The program used the standard random module for driver selection.

These findings were reviewed and addressed in the updated implementation.



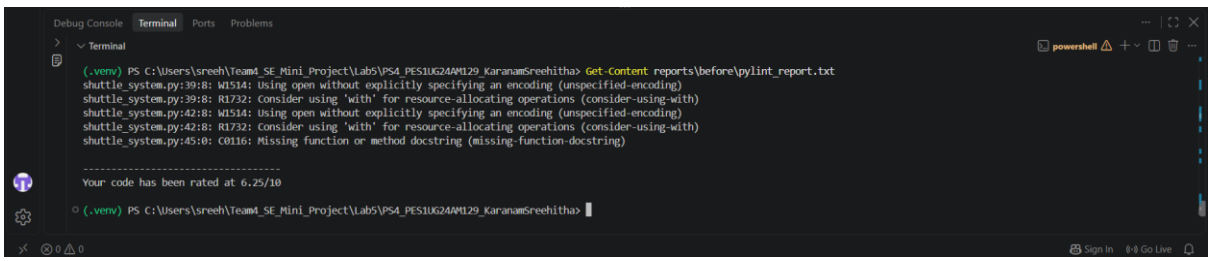
4.3 Flake8 – Initial Formatting Analysis

Flake8 reported 10 formatting and style violations.

The reported issues included:

- E302 – Expected two blank lines before function definitions.
- E305 – Expected two blank lines after class or function definitions.
- E501 – Line too long.
- E722 – Use of bare except statements.

These issues were corrected to improve code readability and maintain consistency with Python style guidelines.



5. Issues Identified and Corrections

The following table summarises the important issues identified during static code analysis and the corrections made.

Tool	Issue	Correction
Pylint	Missing docstrings	Added module and function documentation.
Pylint	Mutable default argument	Replaced the default empty list with None.
Pylint	Old string formatting	Used f-strings.
Pylint	Bare exception handling	Replaced broad exception handling with specific exceptions.
Pylint	Naming convention violation	Renamed assignDriver() to assign_driver().
Pylint	Unused argument	Used the route ID for validation.
Pylint	Global statement usage	Updated the existing route dictionary instead of reassigning it.
Pylint	File handling issues	Used with open() and specified UTF-8 encoding.
Bandit	B110 – Silent exception handling	Removed the exception handler that ignored errors.
Bandit	B311 – Standard random generator	Replaced random.choice() with secrets.choice().
Flake8	E302 and E305	Corrected blank-line spacing.
Flake8	E501	Reformatted long lines.
Flake8	E722	Replaced bare exception handling with specific exceptions.

Additional input validation was introduced to prevent invalid passenger counts and invalid route references. The average-load calculation was also updated to handle routes with no recorded trips.

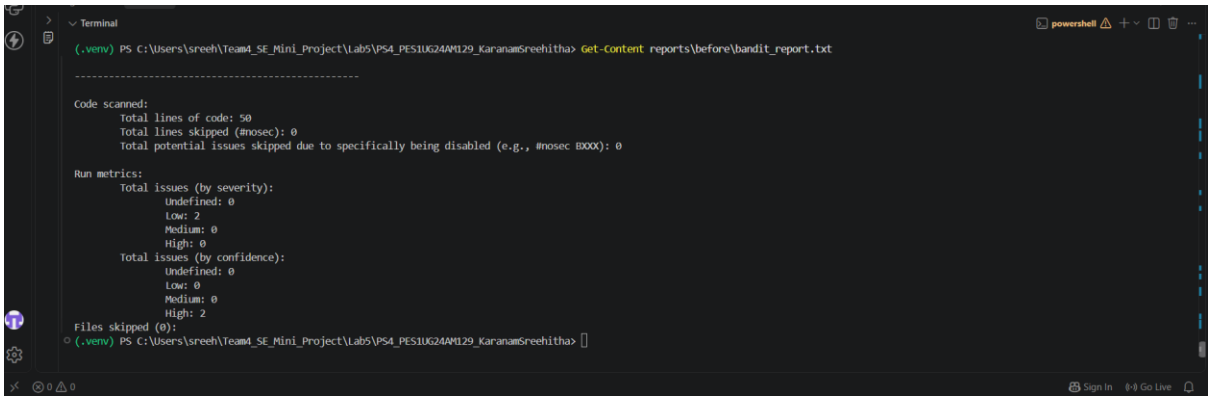
6. Implementation and Corrections

The corrections were implemented in the Python source file shuttle_system.py.

The following improvements were made:

1. Added appropriate documentation to the module and functions.
2. Corrected function naming and string formatting.
3. Removed mutable default arguments.
4. Improved exception handling by using specific exception types.
5. Added validation for passenger counts and route IDs.
6. Updated file operations to use context managers and UTF-8 encoding.
7. Replaced the standard random selection function with secrets.choice().
8. Corrected formatting and spacing issues.
9. Added handling for empty trip lists and removed routes.

The updated program was executed to verify that the corrections did not prevent its expected functionality.



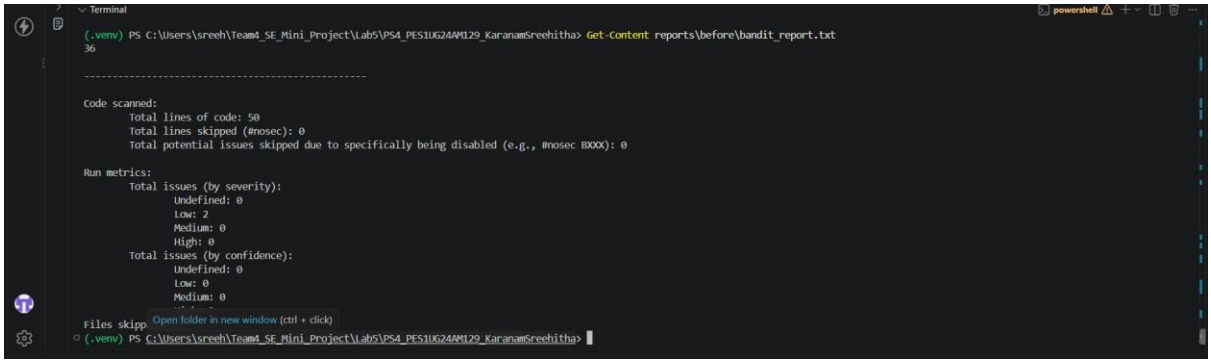
7. Final Analysis and Verification

After implementing the corrections, all three static analysis tools were executed again.

7.1 Final Pylint Results

The updated program achieved a Pylint score of **10.00/10**.

This indicates that the previously identified Pylint issues were resolved.

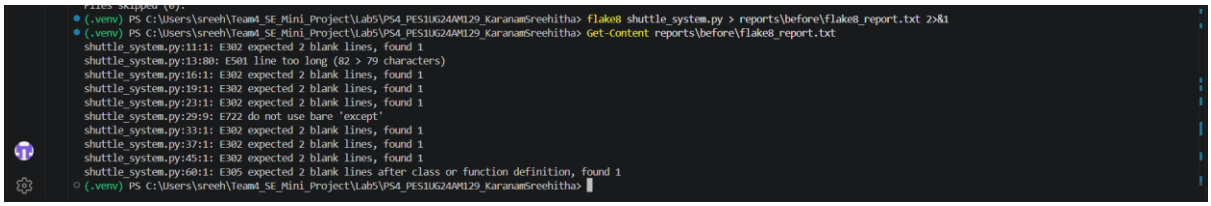


7.2 Final Bandit Results

Bandit reported:

No issues identified.

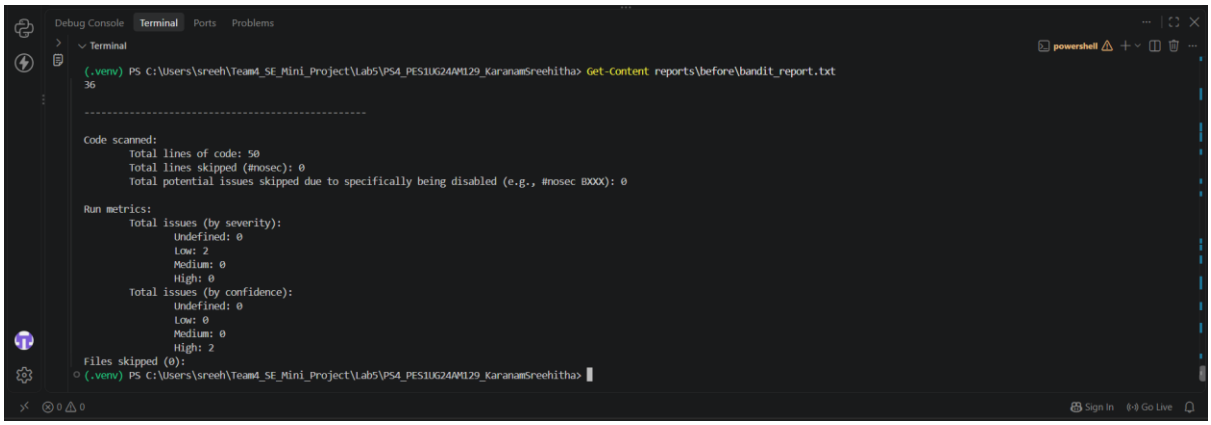
The final report showed zero issues across all severity levels.



7.3 Final Flake8 Results

Flake8 completed without reporting any violations.

This confirms that the previously identified formatting and style issues were corrected.

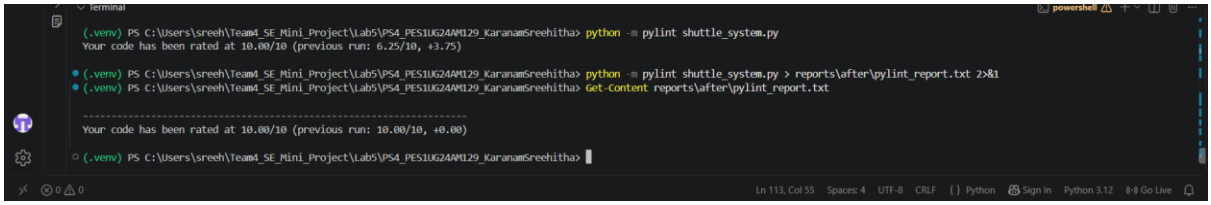


7.4 Summary of Results

Analysis Tool	Initial Result	Final Result
Pylint	6.25/10	10.00/10
Bandit	2 low-severity findings	0 findings

Flake8	10 violations	0 violations

The final results demonstrate an improvement in code quality, security analysis and formatting consistency.



8. Version Control Using Git

Git was used to track the modifications made during the lab. Separate commits were created to maintain a clear record of the corrections associated with the three static analysis tools.

The following commits were created:

1. **Fix Pylint code quality issues**
2. **Fix Bandit security issues**
3. **Complete Flake8 checks and lab documentation**

This approach helped maintain a structured development history and made the changes easier to track.

9. Reflection

1. What did you learn from this lab?

This lab helped me understand the importance of analysing code beyond simply checking whether it executes successfully. Static analysis tools can identify issues that may not be immediately visible during normal execution. I also learned how different tools focus on different aspects of software quality.

2. How did Pylint, Bandit and Flake8 help improve the program?

Pylint helped identify code quality and maintainability issues, Bandit highlighted potential security concerns, and Flake8 helped maintain proper formatting and coding standards. Using all three tools provided a more complete analysis of the program.

3. Why is static code analysis important?

Static code analysis helps developers identify potential problems early in the development process. It improves readability, reduces the possibility of errors, encourages consistent coding practices and makes software easier to maintain.

4. What challenges did you encounter?

The main challenge was understanding the warnings generated by each tool and determining the appropriate corrections without affecting the program's functionality. After making the changes, the program was executed again and all three tools were rerun to verify the results.

5. What is the overall outcome of this lab?

The lab demonstrated how static code analysis can be used to improve an existing Python application. The final program achieved a Pylint score of 10.00/10, Bandit reported no issues, and Flake8 reported no formatting violations.

10. Conclusion

The Static Code Analysis lab was successfully completed using Pylint, Bandit and Flake8.

The original Campus Shuttle System program was analysed, and the issues identified by the three tools were corrected. The updated program was tested again to verify the changes.

The final analysis achieved a Pylint score of 10.00/10, zero Bandit findings and zero Flake8 violations.

Overall, this lab provided practical experience in improving code quality, identifying potential security issues, following coding standards and using Git for version control.